



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



PREVIO N° 01

NOMBRE COMPLETO: COLIMA ESQUIVEL YADIRA

N° de Cuenta: 422115533

GRUPO DE LABORATORIO: 13

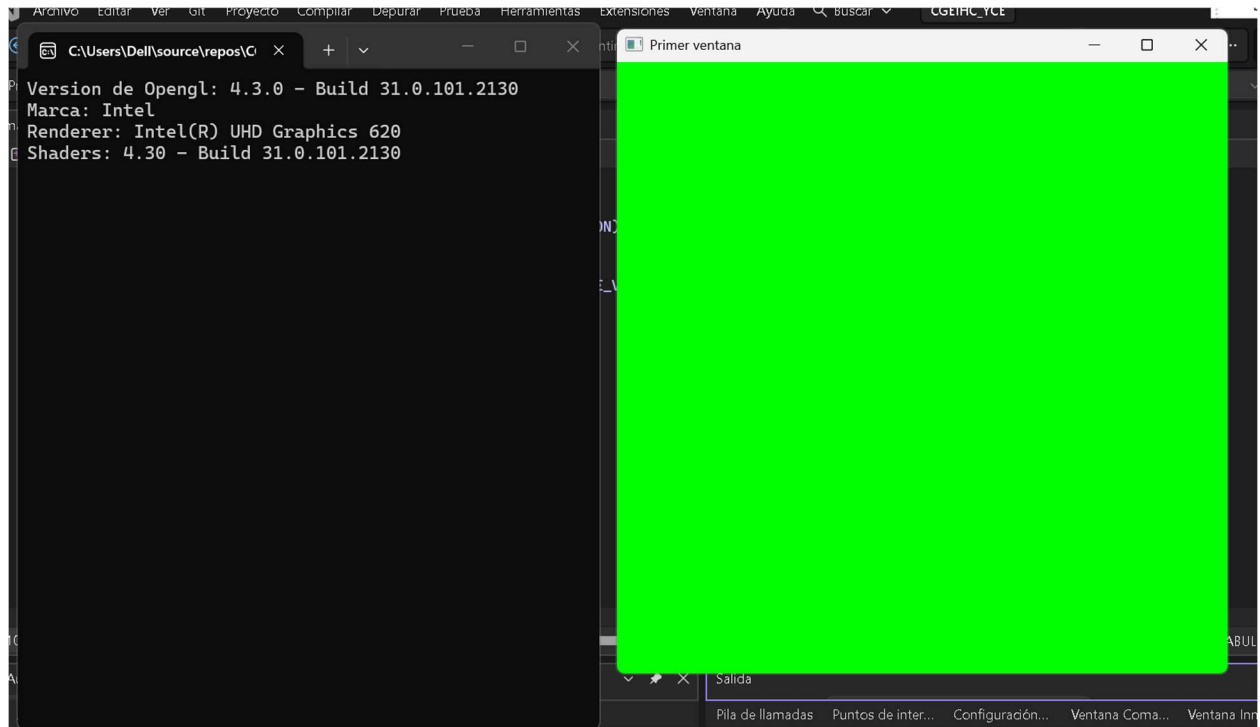
GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 02/09/2026

CALIFICACIÓN: _____

1.-.Captura de pantalla como la del manual de configuración en la cual se muestra la ventana de fondo verde y la información de la consola con los datos de Hardware de su equipo de cómputo (no es necesario que se imprima para la entrega del previo a mano)



2. ¿Qué es un VAO?

Un vao (Vertex Array Object) es un objeto de OpenGL que se utiliza para almacenar y recordar la configuración de los datos de los vértices que se van a utilizar para dibujar un objeto.

Le indica a OpenGL cómo interpretar los datos de los vértices que están almacenados en los VBO, es decir, no guarda los vértices como tal sino información de ellos.

3. ¿Qué es un VBO?

Un VBO o Vertex Buffer Object u Objeto de Búfer de Vértices es un objeto que permite almacenar datos en un bloque de memoria administrado por OpenGL, para posteriormente usarlos durante el proceso de renderizado. Cuando se usa como fuente de datos para los atributos de los vértices, ese buffer se denomina VBO, es decir, es un espacio en memoria donde se almacenan los datos que OpenGL necesita para representar los vértices de un objeto.

- Posición de los vértices
- Color
- Coordenadas de textura
- Normales
- etc.

4. ¿Qué parámetros recibe el comando glVertexAttribPointer?

La función glVertexAttribPointer() le indica a OpenGL cómo debe interpretar los datos de los atributos de los vértices que están almacenados en un VBO, es decir, le dice a OpenGL cómo están organizados los datos.

La función tiene esta estructura

glVertexAttribPointer(index, size, type, normalized, stride, pointer);

- Index: Indica que atributo de vértice se está configurando
- Size: Indica cuántos componentes tiene el atributo.
- Type: Indica el tipo de dato de cada componente.
- Normalized: Indica si los datos de tipo entero deben normalizarse cuando OpenGL los convierte a valores de punto flotante
- Stride: Cuántos bytes hay entre el comienzo de un atributo y el comienzo del mismo atributo en el siguiente vértice
- Pointer: Indica el desplazamiento desde el inicio de VBO hasta donde comienza el atributo

```
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0);
```

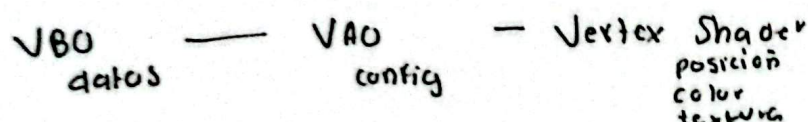
Atributo 0, formado por 3 componentes de tipo float, sin normalización, donde cada vértice ocupa 6 bytes, y los datos comienzan desde el inicio de VBO.

5. ¿Qué información maneja Vertex Shader?

Los sobreadadores de vértices procesan los vértices e indican cuáles son sus coordenadas en el "espacio de recorte", un espacio que facilita a los ordenadores la comprensión de qué vértices son visibles para cámara y cuáles no. Esto hace las GPU sean más rápidas en las etapas posteriores.

Su función principal es transformar y procesar la información de cada vértice y determinar su posición en el espacio de pantalla.

Puede recibir diferentes atributos de vértices dependiendo de lo que necesite el programa. Estos datos normalmente provienen de los VBO, y la configuración de cómo se interpretan se realiza mediante `glVertexAttribPointer()`.

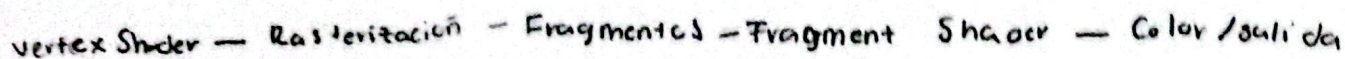


Puede aplicar transformaciones mediante matrices de modelo, vista y proyección. Salida principal es `gl_Position`, que contiene la posición del vértice en coordenadas de clip. También puede generar otras variables de salida que serán utilizadas por las etapas posteriores del pipeline gráfico, como el Fragment Shader.

6. ¿Qué información maneja el Fragment Shader?

Es un programa que se ejecuta en GPU y procesa los fragmentos generados después de la rasterización. Su principal función es determinar el color y otros valores de salida de cada Fragmento antes de que se almacenen en el framebuffer.

Puede recibir información proveniente del vertex Shader, como colores, coordenadas de textura y normales, además de datos mediante variables uniformes y texturas.



7. ¿Qué parámetros recibe el comando `glDrawArrays`?

Renderiza primitivas a partir de datos de matriz

`glDrawArrays` especifica múltiples primitivas geométricas con muy pocas llamadas a subrutinas. En lugar de llamar a un procedimiento GL para pasar cada vértice, normal, coordenada de textura, indicador de borde o color individual, puede preespecificar matrices separadas de vértices, normales y colores y utilizarlos para construir una secuencia de primitivas con una sola llamada a `glDrawArray`.

Parámetros

`glDrawArrays(GLenum mode, first, count)`

mode: Tipo de geometría que se va a utilizar.

first: Índice del primer vértice que se utilizará

count: Número de vértices que se van a utilizar para realizar un trazo, o el dibujo

La función utiliza la configuración establecida mediante el VAO y los datos almacenados en los VBO

8. ¿Cómo funciona `glm::Ortho`?

Crea una matriz de proyección ortográfica, es decir, una vista donde los objetos no se hacen más pequeños al alejarse

uso:

`glm::ortho(left, right, bottom, top, near, far);`

- left: límite izquierdo
- right: límite derecho
- bottom: límite inferior
- top: límite superior
- near: Plano cercano
- far: Plano lejano

`glm::ortho()` transforma las coordenadas de los objetos para proyectarlos sobre la pantalla sin aplicar perspectiva. Por eso, un objeto conserva aproximadamente el mismo tamaño aunque se encuentre más lejos o más cerca.

9. ¿Cómo funciona `glm::Perspective`?

Esta función genera una matriz de proyección en perspectiva que hace que los objetos lejanos se vean más pequeños que los cercanos, simulando la forma en que vemos el mundo.

`glm::perspective(fovy, aspect, near, far);`

fovy: Ángulo de visión vertical

aspect: Relación de aspecto de la ventana,
normalmente ancho/alto

near: Distancia del plano cercano

far: Distancia del plano lejano

Ejemplo:

```
glm::mat4 proyeccion = glm::perspective(
    glm::radians(45.0F),
    800.0F / 600.0F,
    0.1F,
    100.0F
);
```

45° - ángulo campo de visión
800/600 relación de aspecto.
0.1 distancia mínima visible
100 distancia máxima visible.

La matriz transforma las coordenadas de la escena para producir el efecto de profundidad: conforme un objeto se aleja de la cámara, su tamaño aparente disminuye, se usa normalmente junto con las matrices Model y View.

10. ¿Cómo funciona glViewport?

`glViewport()` es una función de OpenGL que define la región de la ventana en la que se realizará el renderizado. Recibe las coordenadas de inicio (x,y) y las dimensiones (width, height) del área de visualización. Permite establecer como se adapta la imagen generada por OpenGL al tamaño de la ventana.

`glViewport(x, y, width, height);`

x: Posición horizontal donde comienza el viewport
y: Posición vertical donde comienza el viewport
width: Ancho del área de renderizado
height: Alto del área de renderizado.

`glViewport(0, 0, 800, 600);`

Significa que OpenGL utilizará un área de 800x600 píxeles, comenzando en la posición (0,0).

11. ¿Cómo funciona glm::translate?

Se usa para mover un objeto dentro de la escena.

Genera una matriz de traslación que permita modificar la posición de un objeto en los ejes x, y, z.

`glm::translate(matriz, desplazamiento)`

matriz: Matriz sobre la que se aplica la traslación

desplazamiento: Vector vec que indica cuánto se moverá el objeto en x, y, z

Ejemplo:

```
glm::mat4 mod = glm::mat4(1.0F);
```

```
* mod = glm::translate(mod, glm::vec3(2.0F, 1.0F, 0.0F));
```

• El objeto se desplaza 2 unidades a la derecha, 1 unidad arriba, no se desplaza en profundidad.

12. ¿Cómo funciona glm::rotate?

Es una función que genera una matriz de rotación para modificar la orientación de un objeto. Recibe una matriz, un ángulo de rotación y un vector que define el eje alrededor del cual se realizará el giro. El ángulo normalmente se proporciona en radianes.

`glm::rotate (matriz, ángulo, eje);`

`glm::mat4 model = glm::mat4 (1.0f);`

```
model = glm::rotate(  
    model,  
    glm::radians(45.0f),  
    glm::vec3(0.0f, 0.0f, 1.0f)  
);
```

El objeto gira 45° alrededor del eje z

13. ¿Cómo funciona glm::scale?

Es una función que genera una matriz de escalamiento para modificar el tamaño de un objeto en los ejes X, Y y Z. Recibe una matriz y un vector vec3 que determina el factor de escala aplicado en cada eje.

`glm::scale (matriz, escala);`

`glm::mat4 model = glm::mat4 (1.0f);`

```
model = glm::scale(  
    model,  
    glm::vec3 (2.0f, 2.0f, 2.0f)  
);
```

El objeto aumenta el doble de su tamaño en los 3 ejes.

14. ¿Qué son las variables uniform dentro de GLSL y cómo se declaran y se mandan desde OpenGL a GLSL?

Es una variable uniform es una variable cuyo valor puede ser establecido desde el programa de OpenGL. Su valor permanece constante durante el procesamiento de un conjunto de primitivas, por lo que, es útil para enviar datos como matrices, colores, posiciones de luces.

Para declararlas en GLSL se usan la palabra `uniform` seguido del tipo y nombre

Uniform mat4 modelo

```
void main() { gl_Position = modelo * vec4(Posición, 1.0); }
```

Primero se obtiene la ubicación de la variable dentro del programa shader:

```
GLint locModelo = glGetUniformLocation(shader, "modelo");
```

Después se envía el valor usando la función `glUniform` correspondiente al tipo de dato

C++ $\rightarrow$ `glGetUniformLocation()` $\rightarrow$ `glUniform()` $\rightarrow$ Uniform en GLSL.

15. ¿Cómo funciona dentro de GLSL `gl_Position`?

Es una variable predefinida de tipo `Vec4` utilizada como salida del Vertex Shader. Contiene la posición de cada vértice en coordenadas de clip y es necesaria para que OpenGL pueda determinar donde se ubicará dicho vértice durante el proceso de renderizado. Normalmente se obtiene aplicando las matrices de modelo, vista y proyección a la posición original del vértice.

```
void main() { gl_Position = vec4(position, 1.0); }
```

```
gl_Position = proyección * vista * modelo * vec4(position, 1.0);
```

- modelo: posición, rotación y escala del objeto.
- vista: posición y orientación de la cámara
- proyección: perspectiva o proyección ortográfica
- `gl_Position`: posición final del vértice para las siguientes etapas del pipeline.

16. ¿Cómo funciona la variable `extern`?

Es una palabra clave C/C++ que permite declarar una variable que está definida en otra parte del programa, generalmente en otro archivo. Indica que la variable existe y puede usarse, pero que su almacenamiento y definición se encuentra en otro lugar. Esto permite compartir variables globales entre diferentes archivos de un proyecto.

```
// archivo1.cpp
```

```
int contador = 10;
```

```
// archivo.cpp
```

```
extern int contador;
```

```
printf("%d", contador);
```

Conclusion

Algunos de los conceptos me parecieron bastante intuitivos y se facilitó comprenderlos pero la mitad de ellos no entendí por completo la idea. Pero ya con todo lo que investigué para el previo le entendí mucho más a la práctica y ahorita que volví a checarlo, por que, sabía que las funciones estaban ahí y sabía o entendía más o menos como funcionaban para ahora puedo entenderlas mejor que parámetros reciben y lo que retornan.

BIBLIOGRAFÍA

Khronos Group. (2023). *The OpenGL Shading Language, Version 4.60.8*. Khronos Group.

Khronos Group. (s. f.). *Vertex specification*. OpenGL Wiki.

Khronos Group. (s. f.). *glDrawArrays*. OpenGL Wiki.

Khronos Group. (s. f.). *OpenGL 4.6 reference pages*.

Khronos Group. (s. f.). *OpenGL Reference*. OpenGL Wiki.

G-Truc Creation. (s. f.). *OpenGL Mathematics (GLM): API documentation*.

G-Truc Creation. (s. f.). *GLM_EXT_matrix_transform*. OpenGL Mathematics.

G-Truc Creation. (s. f.). *GLM_EXT_matrix_clip_space*. OpenGL Mathematics.

Kessenich, J., Sellers, G., & Shreiner, D. (2017). *OpenGL programming guide: The official guide to learning OpenGL, version 4.5 with SPIR-V* (9th ed.). Addison-Wesley Professional.

cppreference.com. (s. f.). *C++ keyword: extern*.